Course: BTech **Semester:** 2

Prerequisite: Basic understanding of programming logic and computer fundamentals.

Rationale : The course aims to introduce the fundamentals of Python programming while enabling students to understand its syntax, control structures, and data structures. It further explores object-oriented and modular programming concepts, along with file handling, exceptions, and built-in functions. The course also focuses on developing problem-solving skills through algorithmic thinking using Python.

Teaching and Examination Scheme

Teaching Scheme					Examination Scheme					Total
Lecture Hrs/Week	Tutorial Hrs/Week	Lab Hrs/Week	Seminar Hrs/Week	Credit	Internal Marks			External Marks		
					T	CE	P	T	P	
3	-	2	-	4	20	20	20	60	30	150

SEE - Semester End Examination, **T** - Theory, **P** - Practical

Course Content

W - Weightage (%) , **T** - Teaching hours

Sr.	Topics	W	T
1	**Introduction to Python** History and features of Python; Installation and execution environment; Python syntax and semantics; Variables, identifiers, keywords; Data types and type conversions; Input/output functions; Operators and expressions	15	7
2	**Control Structures and Functions** Conditional statements: if, if-else, if-elif-else; Looping structures: while, for, break, continue, pass; Functions: definition, calling, return statement; Function arguments, recursion; Lambda functions, map(), filter(), reduce()	21	10
3	**Strings, Lists, Tuples, and Dictionaries** String operations and methods; Lists: indexing, slicing, methods; Tuples: creation, operations, immutability; Sets and frozensets; Dictionaries: key-value access, methods; Comprehensions: list, set, and dictionary	18	9
4	**Object-Oriented Programming in Python** Introduction to OOP; Classes and objects; __init__() method and self parameter; Encapsulation, inheritance, and polymorphism; Special methods and operator overloading; Class vs instance variables and methods	23	11
5	**File Handling, Modules, and Exception Handling** File operations: open, read, write, append, with; Working with text and binary files; Exception handling: try, except, finally, raise; Built-in exceptions; Importing and creating modules; Python standard libraries (os, sys, math, datetime)	23	11
	Total	**100**	**48**

Reference Books

1.	**Python Programming (TextBook)** By by Reema Thareja | Oxford University Press
2.	**Core Python Programming - Second Edition (TextBook)** By Wesley J. Chun | Prentice Hall
3.	**PyFlo - A Free, Interactive Guide to Python Programming BY Benjamin Dicken**
4.	**Python Programming: An Introduction to Computer Science** By John Zelle | Course Technology Cengage Learning Publications | 2nd, Pub. Year 2013
5.	**Python Programming for Absolute Beginers** By Michel Dawson | Course Technology Cengage Learning Publications | 3rd, Pub. Year 2013

Course Outcome

After Learning the Course the students shall be able to:

1. Explain the structure, syntax, and semantics of Python programming language.
2. Apply control structures and functions to solve real-world problems.
3. Analyze and use built-in data types and data structures effectively.
4. Describe the principles of object-oriented programming in Python.
5. Demonstrate proficiency in handling files, exceptions, and modules.

List of Practical

1.	**Install Python and set up different IDEs. Basics of Python Programming** 1. Write a program to print "Hello, World!". 2. Write a program to demonstrate the use of variables and data types. 3. Write a program to perform basic arithmetic operations. 4. Write a program to illustrate type casting between int, float, string.
2.	**Input/Output and Operators** 1. Write a program to take input from the user and display it. 2. Write a program to create a calculator for +, -, *, /, % operations. 3. Write a program to demonstrate comparison and logical operators. 4. Write a program to illustrate use of assignment and compound assignment operators.
3.	**Conditional Statements** 1. Write a program to check whether a number is even or odd. 2. Write a program to find the largest of three numbers. 3. Write a program to check whether a year is a leap year. 4. Write a program to display grades using if-elif-else.
4.	**Loops (for, while)** 1. Write a program to print numbers from 1 to 10 using a loop. 2. Write a program to display multiplication table of a number. 3. Write a program to sum of digits of a number using while loop. 4. Write a program to display factorial of a number using for loop.
5.	**Functions and Recursion** 1. Write a function to add two numbers. 2. Write a program to find factorial using Recursive function. 3. Write a program to create a function to check prime number. 4. Write a program to calculate the square a number using lambda function.
6.	**Strings and String Operations** 1. Write a program to count the number of vowels in a string. 2. Write a program to reverse a string using slicing and loop. 3. Write a program to check whether a string is palindrome. 4. Write a program to convert string to uppercase/lowercase.
7.	**Lists and Tuples** 1. Write a program to create a list and perform append, insert, delete. 2. Write a program to find max, min, and sort elements in a list. 3. Write a program to slice and iterate through a tuple. 4. Write a program to convert list to tuple and vice versa.
8.	**Dictionaries and Sets** 1. Write a program to create a dictionary to store student details. 2. Write a program to add, update, delete dictionary keys. 3. Write a program to create a set and perform union, intersection. 4. Write a program to check for duplicate elements using set.

9.	**File Handling** 1. Write a program to create a text file and write content into it. 2. Write a program to read content from a text file. 3. Write a program to count the number of lines, words, and characters in a file. 4. Write a program to copy the contents from one file to another.
10.	**Exception Handling and Modules** 1. Write a program to handle the divide-by-zero exception. 2. Write a program to illustrate try-except-else-finally blocks. 3. Write a program to import and use the built-in math module. 4. Write a program to create a user-defined module and use it.